



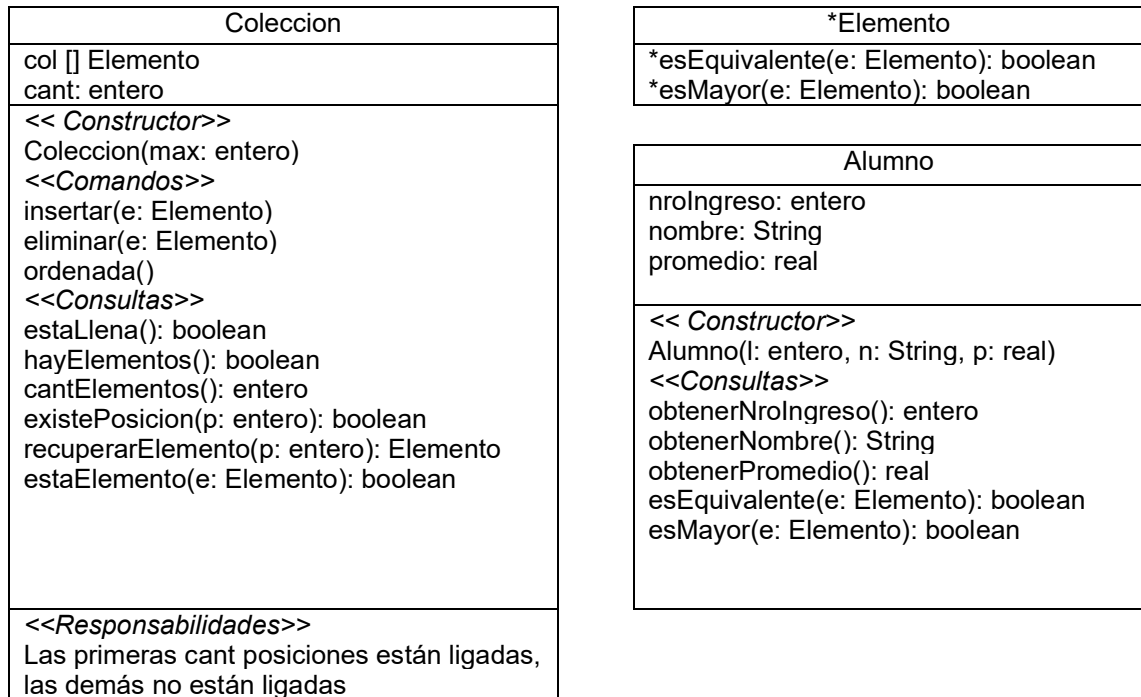
Actividades prácticas: Genericidad

Objetivos del práctico:

- Introducir la noción de genericidad como estrategia para favorecer la productividad.
- Reforzar los conceptos de herencia, polimorfismo y vinculación dinámica.

EJERCICIO 1: Defina el concepto de genericidad.

EJERCICIO 2: Dado el siguiente diagrama en el cual la clase *Alumno* extiende a *Elemento*:



En *Coleccion*:

- *Coleccion(max: entero)*. Crea una colección para *max* elementos. Requiere *max* > 0.
- *insertar(e: Elemento)*. Inserta el elemento *e* en la primera posición libre e incrementa *cant*. Asume que la estructura no está llena y *e* está ligada y no hay un elemento equivalente en la colección.
- *eliminar(e: Elemento)*. Busca un elemento equivalente a *e*. Si existe y ocupa la posición *p*, asigna cada elemento de la posición *i*, con $p < i < cant$, a la posición *i-1* y decrementa *cant*.
- *ordenada()*. Aplica el método *shakesort* para reordenar los elementos de la colección.
- *estaLlena()*: boolean. Retorna *true* si *cant* es igual a la cantidad de elementos del arreglo.
- *hayElementos()*: boolean. Retorna *true* si *cant* > 0.
- *cantElementos()*: entero. Retorna *cant*.
- *existePosicion(p: entero)*: boolean. Retorna *true* si $0 \leq p < cant$
- *recuperarElemento(p: entero)*: Elemento. Retorna el elemento asignado a la posición *p*, si *p* no es válida retorna nulo.
- *estaElemento(e: Elemento)*: boolean. Retorna *true* si algún elemento de la colección es equivalente a *e*, asume *e* ligada.

En *Alumno*:

- *esEquivalente(e: Elemento)*: boolean. Compara el estado interno del alumno que recibe el mensaje y el que pasa como parámetro. Requiere que *e* sea de clase *Alumno*.



- *esMayor(e: Elemento): boolean*. Retorna *true* si el legajo del alumno que recibe el mensaje es mayor al legajo del alumno que pasa como parámetro. Requiere que *e* sea de clase *Alumno*.
- a) Implemente en Java las clases especificadas.
- b) Implemente un tester para verificar los servicios de la clase *Coleccion*.

EJERCICIO 3: Implemente y verifique la clase *ColeccionOrdenada*, asociada a la clase *Elemento* definida antes, de acuerdo a la especificación:

ColeccionOrdenada
t [] Elemento cant: entero
<<Constructores>> ColeccionOrdenada(max: entero) <<Comandos>> insertar(e: Elemento) eliminar(e: Elemento) <<Consultas>> estaLlena(): boolean hayElementos(): boolean cantElementos(): entero existePosicion(p: entero): boolean recuperarElemento(p: entero): Elemento estaElemento(e: Elemento): boolean intercalar(c: ColeccionOrdenada): ColeccionOrdenada
<<Responsabilidades>> Las primeras <i>cant</i> posiciones están ligadas, las demás no están ligadas. La colección está ordenada de acuerdo a la relación de orden establecida en la clase específica de las componentes.

- *Coleccion(max: entero)*. Crea una colección para *max* elementos. Requiere *max* > 0.
- *insertar(e: Elemento)*. Inserta el elemento *e* manteniendo el orden la estructura e incrementa *cant*. Asume que la estructura no está llena y *e* está ligada y no hay un elemento equivalente en la colección.
- *eliminar(e: Elemento)*. Busca un elemento equivalente a *e*. Si existe y ocupa la posición *p*, asigna cada elemento de la posición *i*, con *p* < *i* < *cant* a la posición *i-1* y decrementa *cant*.
- *estaLlena(): boolean*. Retorna *true* si *cant* es igual a la cantidad de elementos del arreglo.
- *hayElementos(): boolean*. Retorna *true* si *cant* es mayor a 0.
- *cantElementos(): entero*. Retorna *cant*.
- *existePosicion(p: entero): boolean*. Retorna *true* si $0 \leq p < cant$.
- *recuperarElemento(p: entero): Elemento*. Retorna el elemento asignado a la posición *p*, si *p* no es válida retorna *nulo*.
- *estaElemento(e: Elemento): boolean*. Retorna *true* si algún elemento de la colección es equivalente a *e*, asume *e* ligada. Aplica búsqueda binaria.
- *intercalar(c: ColeccionOrdenada): ColeccionOrdenada*. Retorna una colección generada intercalando ordenadamente y en un único recorrido, la colección que recibe el mensaje y la que pasa como parámetro.



EJERCICIO 4: Dadas las clases *Coleccion* y *Elemento* definidas antes y, además las clases *Empleado* que extiende a *Elemento* y *NominaEmpleados* que extiende a *Coleccion*:

Empleado	NominaEmpleados
<<Atributos de instancia>> legajo: entero nombre: String cantHoras: entero valorHora: real	<<Constructor>> NominaEmpleados(n: entero) <<Consultas>> contarSupHoras(h: entero): entero sumarSueldos(): real
<<Constructor>> Empleado(leg: entero, nombre: String; canth: entero, valorh: real) <<Consultas>> obtenerLegajo(): entero obtenerSueldo(): real obtenerCantHoras(): entero obtenerValorHoras(): real obtenerNombre(): String esEquivalente(e: Elemento): boolean esMayor(e: Elemento): boolean	Un objeto de clase <i>NominaEmpleados</i> siempre recibe el mensaje insertar con un objeto de clase <i>Empleado</i> como parámetro real.

En *Empleado*:

- *obtenerSueldo(): real*. Computa la cantidad de horas por el valor de la hora.
- *esEquivalente(e: Elemento): boolean* y *esMayor(e: Elemento): boolean*. Comparan el legajo. Requieren que *e* sea de clase *Empleado*.

En *NominaEmpleados*:

- *contarSupHoras(h: entero): entero*. Cuenta la cantidad de empleados que trabajan más de *h* horas.
- *sumarSueldos(): real*. Computa la sumatoria de todos los sueldos

- Implemente la clase *Empleado* extendiendo *Elemento* y la clase *NominaEmpleados* extendiendo a *Coleccion* de acuerdo al diagrama.
- Implemente la clase *Tecnico* extendiendo *Empleado* y redefiniendo el método *obtenerSueldo()* computando el sueldo de cualquier empleado más un 15%.
- Defina una clase tester que cree una nómina de empleados, inserte 12 Empleados, algunos de clase *Tecnico* y muestre la suma de sus sueldos.



EJERCICIO 5: Dado el siguiente diagrama de clases y la especificación de los servicios:

MatrizGenerica
<<Atributos de instancia>> m: Elemento[][]
<<Constructor>> MatrizGenerica(nf, nc: entero) <<Comandos>> establecer(f, c: entero, e: Elemento) <<Consultas>> obtener(f, c: entero): Elemento cantFilas(): entero cantColumnas(): entero pertenece(e: Elemento): boolean contarElem(e: Elemento): entero esEquivalente(c: MatrizGenerica) : boolean esComplemento(c: MatrizGenerica): boolean clone(): MatrizGenerica

Elemento*
<<Consultas>> *equals(e: Elemento): boolean *complemento(): Elemento *clone(): Elemento

Imagen
<<Constructor>> Imagen (n: entero) <<Consultas>> todosGrisen(): boolean Un objeto de clase Imagen siempre recibe el mensaje establecer con un objeto de clase Color como tercer parámetro real.

- *esComplemento(c: MatrizGenerica): boolean*. Retorna *true* si cada elemento de la matriz que recibe el mensaje es equivalente al complemento del elemento que ocupa la misma fila y columna en la matriz que pasa como parámetro.
 - *clone(): MatrizGenerica*. Genera un nuevo objeto de clase *MatrizGenerica* tal que el elemento de cada posición es un clone del elemento que ocupa la misma posición en la matriz que recibe el mensaje.
- a) Implemente la clase genérica *MatrizGenerica*.
- b) Recupere la clase *Color* implementada antes y modifique el código de modo que extienda a la clase *Elemento* y además cambie la definición e implementación de los métodos *complemento*, *clone* e *equals* con la siguiente signatura:
- *complemento(): Elemento*
 - *equals(p: Elemento): boolean*. Requiere que *p* sea de clase *Color*
 - *clone(): Elemento*
- c) Implemente una clase *Imagen* que extienda a la clase *MatrizGenerica* y brinde un método *todosGrisen()* que decida si todos los elementos de la matriz representan algún matiz del color gris.
- d) Implemente una clase tester que cree un objeto de clase *Imagen* y verifique *todosGrisen* y *esComplemento*.



Ejercicio 6: Dado el siguiente diagrama:

Conjunto	Elemento*
<<Atributos de instancia>> ce: Elemento[] cantElementos: entero	*esEquivalente(e: Elemento): boolean
Conjunto(max: entero) <<Comandos>> insertar(e: Elemento) <<Consultas>> pertenece(e: Elemento): boolean vacio(): boolean incluido(c: Conjunto): boolean esEquivalente(c: Conjunto): boolean disjunto(c: Conjunto): boolean union(c: Conjunto): Conjunto interseccion(c: Conjunto): Conjunto diferencia(c: Conjunto) : Conjunto cardinalidad(): entero	

En la clase *Conjunto*

- *Conjunto(max: entero)*. Crea un conjunto vacío que puede tener hasta *max* elementos.
 - *insertar(e: Elemento)*. Inserta un elemento al final e incrementa *cantElementos*. Si el elemento ya existe insertar no tiene ningún efecto.
- a) Implemente la clase genérica *Conjunto* usando un arreglo.
- b) Implemente una clase tester que verifique los servicios provistos por la clase *Conjunto*, a partir de dos conjuntos cuyas componentes sean de la clase *Punto* implementada antes.

Para dos objetos *A* y *B* de clase *Conjunto*, el tester debe verificar que:

A.esEquivalente(B) es verdadero sí y solo sí *A.incluido(B)* y *B.incluido(A)* son verdadero.

A.esEquivalente(B) es verdadero sí y solo sí *A.diferencia(B)* y *B.diferencia(A)* son el conjunto vacío.

A.disjunto(B) es verdadero sí y solo sí *A.interseccion(B)* es el conjunto vacío.

A.disjunto(B) es verdadero sí y solo sí la cardinalidad de *A.union(B)* es igual a la cardinalidad de *A* más la cardinalidad de *B*.

A.union(B) es igual a la unión de *A.diferencia(B)*, *A.intersección(B)* y *B.diferencia(A)*.



EJERCICIO 7. Dado el siguiente diagrama de clase que modela una clase genérica *Conjunto* y la clase *Criatura* que hereda de la clase *Elemento*:

Conjunto	Criatura
<<Atributos de instancia>> co: Elemento[] cant: entero	<<Atributos de clase>> maxEnergia = 1000 <<Atributos de instancia>> nombre: String energia: entero
<<Constructor>> Conjunto(max: entero) <<Comandos>> insertar(e: Elemento) <<Consultas>> pertenece(e:Elemento):boolean interseccion(c: Conjunto): Conjunto aLoSumoNComplemento(n:entero,e:Elemento): boolean cantElementos(): entero	<<Constructor>> Criatura(nom: String) <<Comandos>> jugar() <<Consultas>> obtenerNombre(): String obtenerEnergia(): entero equals(c: Elemento): boolean complemento(c: Elemento): boolean
	Tribu <<Constructor>> Tribu(max: entero) <<Consultas>> hayTresDecrecientes():boolean <<Responsabilidades>> Las clases clientes solo insertan objetos de clase <i>Criatura</i> .
<<Responsabilidades>> Garantiza que la estructura no mantiene dos elementos equivalentes, las primeras <i>cant</i> posiciones están ligadas y las demás no están ligadas.	*Elemento *equals(c: Elemento): boolean *complemento(c: Elemento): boolean

a) Implemente la clase *Conjunto* considerando las siguientes funcionalidades y responsabilidades:

- *Conjunto(max: entero)*. Requiere *max* > 0
- *insertar(e: Elemento)*. Asigna *e* a la posición *cant* e incrementa el valor de esta variable.
- *pertenece(e: Elemento):boolean*. Retorna *true* si el conjunto mantiene un elemento equivalente a *e*.
- *interseccion(c: Conjunto):Conjunto*. Genera un nuevo conjunto que incluye a los elementos que están en la intersección entre el conjunto que recibe el mensaje y el conjunto *c*, siempre comparando por estado interno.
- *aLoSumoNComplemento(n:entero,e:Elemento):boolean*. Retorna *true* si el conjunto que recibe el mensaje tiene a lo sumo *n* elementos que son el complemento de *e*.

b) Implemente la clase *Elemento* y la clase *Criatura* que extiende a *Elemento*.

- *Criatura(nom: String)*. Requiere *nom* ligado, inicializa el atributo energía con el valor del atributo de clase.
- *jugar()*. Consume 1 unidad de energía, siempre y cuando este valor sea mayor a 0.
- *equals(c:Elemento):boolean*. Compara en forma superficial.
- *complemento(c:Elemento):boolean*. Retorna *true* si la energía de la criatura que recibe el mensaje sumada a la energía de *c* es un valor mayor al atributo de clase.



- c) Implemente la clase *Tribu* que extiende a *Conjunto*.
- *Tribu(max: entero)*. Requiere $max > 0$.
 - *hayTresDecrecientes():boolean*. Retorne true si el conjunto contiene una y solo una secuencia de exactamente tres criaturas que ocupan posiciones consecutivas y tienen valores decrecientes de energía.
- d) Implemente una clase tester que valide los métodos *insertar* y *hayTresDecrecientes*.

EJERCICIO 8. Dadas las siguientes definiciones de clases y las declaraciones de variables

<pre>class Alfa { int f () { return 1; } int g () { return 2; } } class Beta extends Alfa { int f () { return 5; } int h () { return 10; } }</pre>	<pre>class Delta { int v1 (Alfa a) { return a.f()*a.f(); } int v2 (Alfa a) { return a.g()*a.g(); } } class Gama extends Delta { int v1 (Beta a) { return a.f()*a.h(); } int v2 (Alfa a) { return a.f()*a.g(); } }</pre>
--	---

```
Alfa a1 = new Alfa();
Alfa a2 = new Beta();
Beta b = new Beta();
Delta d1 = new Delta();
Delta d2 = new Gama();
Gama g = new Gama();
```

Muestre cuál sería la salida por consola:

<pre>System.out.println(d1.v1(a1)); System.out.println(d1.v2(a1)); System.out.println(d1.v1(a2)); System.out.println(d1.v2(a2)); System.out.println(d1.v1(b)); System.out.println(d1.v2(b));</pre>
<pre>System.out.println(d2.v1(a1)); System.out.println(d2.v2(a1)); System.out.println(d2.v1(a2)); System.out.println(d2.v2(a2)); System.out.println(d2.v1(b)); System.out.println(d2.v2(b));</pre>
<pre>System.out.println(g.v1(a1)); System.out.println(g.v2(a1)); System.out.println(g.v1(a2)); System.out.println(g.v2(a2)); System.out.println(g.v1(b)); System.out.println(g.v2(b));</pre>